Listen

Learning Doc 04: Local Agent Internals
Audio, threads, events, feedback and timing

Detailed design, Section 3

13 October 2026

Abstract

Docs 01–03 decided *what* the laptop agent does. This document designs *how it runs*: how sound gets from your microphone to a command, which parts work at the same time (threads), how they talk (events and queues), how Listen answers you (chimes, pop-ups, lowering other sounds), how it follows your microphone when you plug in a headset, and how we will *measure* that the whole trip takes under 1.5 seconds.

Contents

1 Decisions this section is built on **2**

2 Doing several things at once: threads and queues **2**

3 The threads of the agent **2**

4 Events: what travels on the bus **3**

5 The listener as a state machine **4**

6 Feedback: chimes, pop-ups and lowering other sounds **4**
6.1 Chimes 4
6.2 Pop-ups 5
6.3 Lowering other sounds (“ducking”) 5

7 Following your microphone **5**

8 The 1.5-second budget **6**
8.1 How we will measure 6

9 Start-up, shut-down and failures **6**

10 What you see in the terminal (v1 shell) **7**

11 Glossary **7**

12 Check your understanding **8**

1 Decisions this section is built on

Topic	Decision
Feedback	Short sounds (chimes) plus a small pop-up at the bottom of the screen. No spoken voice.
Clearer audio	While Listen is capturing a command, other apps' sound is lowered, then restored.
Speed	Under 1.5 s from the end of your sentence to the action. Measure `tiny.en` and `base.en`, then choose.
Microphone	Always the Windows *default* input; follow it when it changes (headset plugged in).

2 Doing several things at once: threads and queues

The agent must listen to the microphone *continuously*, while also transcribing, opening apps, watching the foreground window and saving to the database. If these ran one after another, the microphone would go deaf every time Listen opened Chrome.

Concept: Thread

A **thread** is one line of work inside a program. A program with several threads can wait for audio in one while it launches an app in another. (Doc 01 introduced the word; here we use it for real.)

Concept: Queue (message passing)

Threads that share variables directly can corrupt each other's data (two threads updating the same list at the same instant). The safe pattern is a **queue**: a thread-safe waiting line. One thread *puts* messages in; another *takes* them out, one at a time, in order. Threads never touch each other's data; they only pass messages. Python's built-in `queue.Queue` does all the locking for us.

Analogy

A restaurant kitchen: the waiter does not walk into the kitchen and stir the pot. They pin an order ticket on the rail (the queue). The cook takes tickets in order. Waiter and cook never get in each other's way, and nothing is lost when the kitchen is busy: tickets just wait.

Concept: The GIL (and why threads are still fine here)

Python has a **Global Interpreter Lock**: only one thread runs *Python* code at a time. That sounds like it defeats threads, but our heavy work happens *outside* Python: speech recognition runs in compiled C++ (inside `faster-whisper`), and audio and Windows calls are also native code. Those release the lock while they work. Most of our threads spend their time *waiting* (for audio, for a queue, for a timer), which costs nothing.

3 The threads of the agent

Each module owns one worker thread and one **inbox** queue. The event bus delivers events into the inboxes of the modules that subscribed to that kind of event.

Thread	Job
Audio capture	Receives 30 ms chunks of sound from the microphone and pushes them into an audio queue. Does nothing else, so no sound is ever dropped.
Listener	Runs voice detection and the wake word on those chunks; decides when a command starts and ends (Section 5); hands finished recordings on.
Transcriber	Turns a recording into text with Whisper. Only one at a time.
Intent	Matches text to a command (Doc 03). Fast, so it also handles confirmations.
Executor	Runs actions and routines; owns the undo slot.
Feedback	Plays chimes, shows pop-ups, lowers and restores other apps' volume.
Usage	Checks the foreground window every 2 s (Section 4 of the design).
Storage	The only thread that writes to SQLite; saves events in small batches.
Housekeeping	Every 2 s: has the default microphone changed? Has a command file changed?
Hotkeys	Push-to-talk key and the continuous-mode on/off key.

Why it is done this way

One writer for the database avoids two threads writing at once (SQLite allows only one writer at a time). **One transcriber** keeps CPU use predictable: if you speak twice quickly, the second sentence simply waits its turn. And **audio capture does nothing but capture**: anything slow there would cause gaps in the sound.

4 Events: what travels on the bus

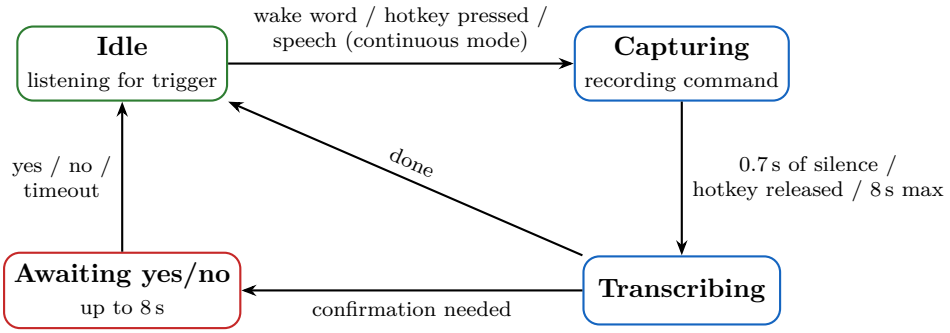
Every event is a small, read-only record (a Python `dataclass`). Each carries the time it was created, so we can measure every stage.

Event	From	Means
`WakeWordDetected`	listener	"Hey Jarvis" heard
`CaptureStarted / Ended`	listener	A command recording began / finished
`UtteranceCaptured`	listener	A finished recording, plus which mode produced it
`Transcribed`	transcriber	The text, its confidence and how long it took
`HeardInContinuous`	transcriber	Text heard in continuous mode (for the private log)
`IntentMatched`	intent	Command + filled slots + match confidence
`IntentNotMatched`	intent	Text + why (no pattern / below threshold)
`ConfirmationRequested`	intent	"Start DSA?": waiting for yes/no
`CommandExecuted`	executor	Success, result, undo information, timings
`CommandFailed`	executor	What went wrong
`UndoPerformed`	executor	What was reversed (or "can't undo")
`ForegroundChanged`	usage	New app and cleaned title in front
`ModeChanged`	hotkeys/intent	Wake word / continuous / off
`MicrophoneChanged`	housekeeping	New default input device
`ModuleError`	any	A module hit an error (it keeps running)

5 The listener as a state machine

Concept: State machine

A **state machine** is a thing that is always in exactly one *state*, and moves between states only on specific *events*. Writing the listener this way makes every situation explicit: "what happens if you press push-to-talk *while* it is waiting for a yes/no?" has one clear answer instead of a tangle of `if` statements.



The three listening modes differ only in *what moves Idle to Capturing*:

- **Wake word**: the wake-word model fires (it only runs on chunks that voice detection marked as speech, which keeps it cheap).
- **Push-to-talk**: the hotkey is pressed; releasing it ends capture.
- **Continuous**: voice detection alone starts capture. The guardrails from Doc 03 (standalone sentence, ≤ 8 words, stricter thresholds) are applied after transcription.

The pre-roll buffer. Voice detection needs a moment to be sure someone is speaking, and you might start talking the instant you finish "Hey Jarvis". So the listener always keeps the last **0.3 s** of audio in a small rolling buffer and adds it to the front of every recording. No first syllable is ever cut off.

Confirmations without a voice. When a command needs confirmation, Listen plays a distinct *question* chime and shows a pop-up: "Start DSA? Say yes or no". For the next 8 seconds it listens *without* needing the wake word. "Yes" runs it, "no" cancels it, silence cancels it (with the "cancelled" chime).

6 Feedback: chimes, pop-ups and lowering other sounds

6.1 Chimes

Four short, clearly different sounds, played with Python's built-in `winsound` (no extra download):

Chime	Meaning
listening	wake word heard / recording started
done	command executed
question	waiting for yes / no
not caught	didn't catch that / failed / can't undo / cancelled

While a chime plays, the listener ignores the microphone for that fraction of a second, so Listen never "hears" its own sound.

Continuous mode is quiet. In continuous mode, every conversation in the room starts a capture, so chiming there would mean a chime (and a "didn't catch that") after almost every sentence anyone says. So in continuous mode Listen plays **no** "listening" chime and gives **no** "not caught" chime or pop-up. Only when a command actually *runs* do you get the "done" chime and its pop-up (and the question chime for the few commands still set to `confirm`, such as closing an app).

6.2 Pop-ups

A small Windows notification at the bottom right: "Searching Google: what is an atom", "Didn't catch that", "Start DSA? Say yes or no". These use the standard Windows notification system, so they appear in the Action Center history too.

6.3 Lowering other sounds ("ducking")

Concept: Audio ducking

Ducking temporarily lowers other sounds so the important one is clear, like a radio DJ's music dipping when they talk. Windows gives every app its own volume *session*, so Listen can lower Spotify and YouTube to 20% without touching the master volume, and put each one back exactly where it was.

- **When:** from the moment capture starts (wake word heard / hotkey pressed) until transcription begins, typically 2–4 seconds.
- **Only other apps**: Listen's own chimes and the master volume are untouched, so "volume up" and its undo keep working normally.
- **Safe restore**: the original levels are written to a small file before ducking. If Listen crashes mid-duck, it restores them from that file on next start. Your music is never left stuck at 20%.

Watch out

Ducking does not happen in continuous mode (decided). In continuous mode, capture starts whenever *anyone* speaks, so ducking there would pump your music down and up during every conversation in the room. Ducking stays for wake-word and push-to-talk, where capture starts only when you asked for it.

7 Following your microphone

Listen always records from the Windows **default input device**. Every 2 seconds the housekeeping thread asks Windows which device that is. If it changed (you plugged in a headset, or Windows switched to Bluetooth earbuds), Listen closes the old audio stream, opens the new one, publishes `MicrophoneChanged`, and shows a pop-up: "Now listening on: Headset Microphone". If no microphone exists at all, Listen waits and retries instead of crashing.

Watch out

Known Windows quirk to verify early: the audio library keeps a cached device list, so a headset plugged in *after* start-up may not appear until the library is restarted. The housekeeping check therefore refreshes the library's device list before comparing. This is one of the first things we test on your laptop, because it is easy to get wrong and annoying when it is.

8 The 1.5-second budget

Concept: Latency budget

Latency is waiting time. A **budget** splits a total target into allowances per stage, like splitting a monthly budget into rent and food. If the total is over, the budget shows *which* stage to fix.

Stage	Budget	Notes
Detect you stopped talking	700 ms	The silence the listener waits for. Starts at 0.7 s (decided); tested at 0.5–0.9 s.
Speech-to-text	500 ms	Now the tightest stage: the number we must measure for `tiny.en` vs. `base.en`.
Matching	<5 ms	Pure function (Doc 03).
Action starts	200 ms	Launching the browser or app; the app's own loading time is not counted.
Feedback	50 ms	Chime + pop-up.
Total	**≈1.45 s**	

Raising the silence wait from 0.5 s to 0.7 s took 200 ms from somewhere: the speech-to-text allowance dropped from 700 ms to 500 ms. That is exactly how a budget works: a total stays fixed, and spending more in one row means spending less in another. If the benchmark shows the model needs more than 500 ms, we choose between a smaller model, a shorter silence, or accepting a total slightly above 1.5 s, and we choose with numbers.

Why it is done this way

Notice that the first row is not the AI model at all: it is *waiting for silence*. Make it too short (200 ms) and Listen cuts you off when you pause mid-sentence ("search… what is an atom"); too long and every command feels sluggish. Push-to-talk skips this wait entirely, because releasing the key *is* the end. Without a budget you would probably try to speed up the model, the wrong place to start.

8.1 How we will measure

1. In **benchmark mode**, speak about 20 of your real commands. No audio is ever saved (Doc 05): each clip stays in memory only long enough to be transcribed.
2. Each clip goes through `tiny.en` *and* `base.en` (both in the fast `int8` format), one after the other, so both models hear identical audio. Only the text results and timings are stored, then the clip is discarded.
3. Pick the smallest model that is both under budget and accurate enough. Expectation (to be verified, not assumed): `base.en` is noticeably more accurate and still fits.

The agent also stamps every event with a time, so the terminal shows the real breakdown of every command you give, for example `silence 700 | stt 420 | match 1 | action 160 | total 1281 ms`.

9 Start-up, shut-down and failures

Start-up order. Load settings → load and validate command files → build the app index → restore any volumes left ducked by a crash → load the speech and wake-word models → open the

microphone → start all threads → "listening" chime and pop-up "Listen is ready". Each step prints a line in the terminal, so a failure points at exactly one step.

Shut-down (Ctrl+C or "quit listening"). Stop accepting audio → let the current command finish → restore ducked volumes → flush unsaved events to SQLite → close.

When something breaks. Every worker thread runs its loop inside an error handler. An error in one module is logged, published as `ModuleError`, shown in the terminal, and that module carries on with the next message. A **supervisor** checks every few seconds that each thread is alive and restarts any that died. One bad command must never take the whole assistant down.

Concept: Graceful degradation

When part of a system fails, the rest keeps working with reduced ability instead of crashing. No internet: commands still work, the command builder falls back to the form. No microphone: Listen waits. Unknown app: "CLion not found", the rest of the routine still runs.

10 What you see in the terminal (v1 shell)

```
[09:41:02] ready  | mic: Realtek Microphone Array | mode: wake word | model: base.en
[09:41:15] wake word
[09:41:17] heard  "search what is an atom"   (confidence 0.93)
[09:41:17] match  search  query="what is an atom"  target=google  (1.00)
[09:41:17] done   open_url google.com/search?q=what+is+an+atom
           timing silence 700 | stt 412 | match 1 | action 140 | total 1253 ms
```

Every line is one event on the bus. When something misbehaves, you will see *which stage* did it, which is the whole point of building the terminal shell first.

11 Glossary

Audio ducking
Temporarily lowering other sounds so one sound is clear.

Dataclass
A Python class that just holds named fields; used for events.

GIL Python's lock letting one thread run Python code at a time; native code releases it.

Graceful degradation
Failing partially instead of completely.

Inbox
The queue in which a module receives its events.

Latency budget
A total time target split into per-stage allowances.

Pre-roll buffer
The last fraction of a second of audio, kept so recordings never start late.

Queue
A thread-safe waiting line for passing messages between threads.

State machine
Something that is always in one state and changes state only on events.

Supervisor
A watcher that restarts parts of a program that died.

Thread
One line of work inside a program.

12 Check your understanding

Check yourself

1. Why does the audio-capture thread do nothing except capture?
2. Two threads both want to save an event at the same moment. How does the design prevent a conflict?
3. In the state machine, what moves Listen from *Idle* to *Capturing* in each of the three modes?
4. Why is ducking switched off in continuous mode?
5. Listen crashes while Spotify is ducked to 20%. What happens on the next start, and why is the original level safe?
6. Your total latency is 2.1 s and the breakdown says `silence 700 | stt 1350`. What do you try first?
7. Why does push-to-talk feel faster than the wake word, even with the same model?